

Proyecto Intermodular

AVTech Invoice.



Salvador Domingo García Sánchez.

Thursday, 30 April 2026

CESUR CFGS Desarrollo de aplicaciones Multiplataforma (Madrid en remoto).

Índice.

Índice.	2
1. <u>Introducción.</u>	4
1.1. <u>Identificación de necesidades y demandas del mercado.</u>	4
1.2. <u>Descripción del proyecto.</u>	4
1.3. <u>Justificación de la solución propuesta.</u>	4
1.4. <u>Características principales.</u>	5
2. <u>Casos de uso funcionalidades.</u>	6
2.1. <u>Descripción de los Casos de Uso Principales:</u>	6
3. <u>Requisitos funcionales y no funcionales.</u>	8
3.1. <u>Requisitos Funcionales (RF)</u>	8
3.2. <u>Requisitos No Funcionales (RNF)</u>	9
4. <u>Tecnologías previstas.</u>	10
4.1. <u>Especificaciones del Stack Tecnológico</u>	10
4.2. <u>Integración con el Ecosistema Google.</u>	12
4.3. <u>Capa de Datos (SQLite + Java).</u>	12
4.4. <u>Generación de Documentos (PDF).</u>	12
4.5. <u>Interfaz Gráfica (JavaFX).</u>	12
5. <u>Boceto inicial Arquitectura.</u>	13
6. <u>Esquema visual.</u>	14
6.1. <u>Esquema de ventana general.</u>	14
6.2. <u>Esquema de la ventana de Login.</u>	15
7. <u>Identidad del proyecto.</u>	16
7.1. <u>Paleta de colores.</u>	16
7.2. <u>Logotipo.</u>	16
7.3. <u>Formato y fuente de texto.</u>	17
7.4. <u>Recursos adicionales.</u>	17
8. <u>Diagrama entidad-relación base de datos .</u>	18
<u>Tablas DB y atributos</u>	19
9. <u>Diseño de la interfaz.</u>	20
10. <u>Diseño de la arquitectura del software.</u>	21
11. <u>Diseño rutas Backend (API REST: endpoints, métodos, recursos).</u>	22
11.1. <u>Diseño de Servicios y Acceso a Datos (Lógica de Negocio y DAOs)</u>	22

11.2.	Módulo de Autenticación y Gestión de Usuarios (UsuarioDAO y GoogleAuthService).	22
11.3.	Módulo de Clientes (ClienteDAO).	22
11.4.	Módulo de Calendario (GoogleCalendarService).	23
11.5.	Módulo de Facturación (FacturaDAO y FacturaPDFService).	23
11.6.	Gestión Centralizada de la Conexión (DatabaseConnection.java).	23
12.	Diseño componentes Frontend.	24
12.1.	Estructura de Contenedores (Layout Principal).	24
12.2.	Componentes de Vista (Módulos).	24
12.3.	Componentes de Soporte (Lógica del Cliente).	25
13.	Fase de Desarrollo.	27
13.1.	Enlace a proyecto GitHub.	27
13.2.	Enlace al video demostrativo de funcionamiento.	27
13.3.	Creación de un nuevo usuario.	27
13.4.	Registro de App y suscripción a API Calendar, generando el archivo credentials.json.	28
13.5.	Proceso de Login	29
13.6.	Ventana cliente.	30
13.7.	Generación de Facturas	31
13.8.	Base de datos.	32
13.9.	Empaquetado y Despliegue (Fat JAR y Launcher).	35
13.10.	Creación de Instaladores Nativos (.dmg y .exe) para MacOS y Windows.	35
13.11.	Fase de Pruebas.	36
13.12.	Cambios y problemas en la fase de desarrollo	37
13.13.	Conclusiones.	38
13.14.	Futuras Líneas de Desarrollo.	39

1. Introducción.

1.1. Identificación de necesidades y demandas del mercado.

AVTech Invoice nace para dar respuesta a una problemática crítica entre los técnicos del sector audiovisual y otros profesionales autónomos: la elevada carga administrativa derivada de la facturación manual. Actualmente, la gestión de facturas puede consumir entre uno y dos días laborables al cierre de cada mes, lo que representa una pérdida de productividad y un coste de oportunidad significativo.

Aunque existen soluciones consolidadas en el mercado como ContaSimple, Billin o Holded, AVTech Invoice introduce una ventaja competitiva diferencial: la automatización basada en el flujo de trabajo real del profesional. Al integrarse directamente con Google Calendar (herramienta estándar para la gestión de reservas y disponibilidad en el sector), el software elimina la necesidad de introducir datos manualmente, transformando las reservas de tiempo en documentos contables de forma casi instantánea.

1.2. Descripción del proyecto.

AVTech Invoice es una aplicación de escritorio diseñada para la gestión automatizada de la facturación por clientes y jornadas de trabajo. El software está especializado en las dinámicas del sector audiovisual, donde los profesionales suelen gestionar múltiples clientes y proyectos distribuidos en un calendario anual.

El funcionamiento del sistema se basa en la sincronización de los calendarios de Google del usuario. La aplicación procesa los eventos y reservas vinculados a cada cliente para generar, de manera automática, facturas en formato PDF. El sistema desglosa los servicios prestados, el total de días trabajados y, si se requiere, el número de proyecto asociado. Esta extracción de datos se realiza a partir de la descripción de los eventos en el calendario, permitiendo que, tras una configuración correcta, la factura se genere sin intervención manual, aunque manteniendo siempre la posibilidad de edición y supervisión final por parte del usuario.

1.3. Justificación de la solución propuesta.

La elección de este tipo de proyecto se fundamenta en la necesidad de centralizar y automatizar el ciclo de facturación. Al vincular la contabilidad directamente con la planificación temporal (Google Calendar), se minimizan los errores humanos y se reduce drásticamente el tiempo dedicado a tareas burocráticas, permitiendo al autónomo centrarse en su actividad principal.

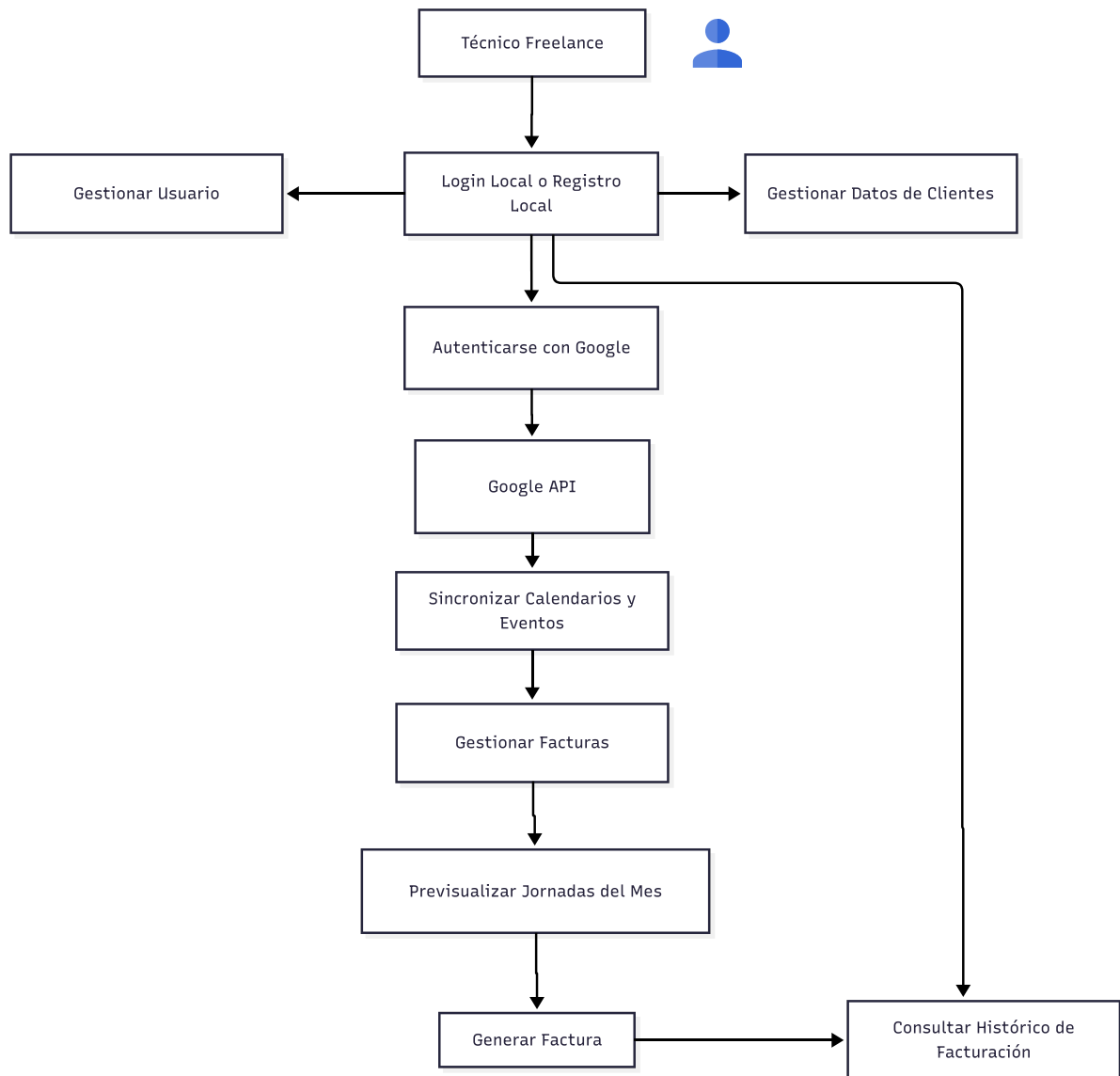
1.4. Características principales.

Para cumplir con sus objetivos, AVTech Invoice integra las siguientes funcionalidades clave:

- **Autenticación segura:** Login seguro con encriptación de contraseña local. Integración con la API de Google para el acceso y lectura de calendarios del usuario.
- **Sincronización de eventos:** Capacidad para filtrar y organizar reservas de fechas por cliente y proyecto.
- **Personalización:** Sistema de plantillas predefinidas que aseguran que las facturas cumplan con los requisitos estéticos y legales del profesional.
- **Generación automatizada:** Procesamiento de datos del calendario para la creación inmediata de facturas exportables en PDF.

Nota: Las funciones de generación automática y configuración de plantillas (Tabla Opciones) forman parte de la arquitectura base preparada para futuras actualizaciones.

2. Casos de uso funcionalidades.



2.1. Descripción de los Casos de Uso Principales:

- **Iniciar Sesión:** El usuario accede a la aplicación utilizando su usuario local creado, posteriormente inicia sesión con su cuenta de Google. El sistema se comunica con la API de Google para validar la identidad y obtener los permisos necesarios.
- **Sincronizar Eventos:** El sistema lee los calendarios del usuario. Filtra aquellos que coinciden con los nombres de los clientes registrados y extrae las fechas trabajadas.
- **Gestionar Clientes:** Interfaz para que el usuario pueda añadir información fiscal de sus clientes, editar datos o asociar nombres específicos de calendarios a empresas reales.

- **Generar Factura PDF:** El proceso central. El sistema toma los eventos del mes seleccionado, calcula los importes (basándose en los datos del cliente/tarifas) y genera un archivo PDF profesional.

Nota: Este caso de uso incluye la sincronización de eventos y la consulta de la base de datos de clientes.

- **Gestionar Usuarios:** Permite añadir o modificar datos relativos al usuario como el IBAN para la facturación.
- **Visualizar Histórico:** El usuario consulta la base de datos interna para ver facturas emitidas anteriormente, permitiendo su descarga o reenvío.

Relación con las Bases de Datos (Arquitectura Interna):

Aunque el diagrama de casos de uso se centra en el "qué hace", el sistema gestionará internamente tres almacenes de datos:

- DB Usuarios: Almacena el ID de Google y preferencias de usuario.
- DB Clientes: Información legal, dirección y tarifas por jornada.
- DB Facturas: Registro histórico para control de ingresos y numeración correlativa.

3. Requisitos funcionales y no funcionales.

Para que el proyecto esté bien definido técnicamente, es fundamental separar qué hace el sistema (Requisitos Funcionales) de cómo debe comportarse el sistema (Requisitos No Funcionales).

3.1. Requisitos Funcionales (RF)

Estos definen las funciones específicas que el software debe ejecutar.

- **Gestión de Usuarios y Autenticación:**

- RF1. Autenticación híbrida y segura: El sistema debe permitir el registro e inicio de sesión local utilizando un correo electrónico y contraseña (cifrada mediante algoritmos de hashing fuerte). Una vez autenticado, el sistema permitirá al usuario vincular su cuenta de Google mediante el protocolo OAuth 2.0 desde el panel principal, habilitando así el acceso a la API de Calendar.
- RF2. Perfil fiscal del usuario: El sistema debe permitir al usuario registrar y editar sus propios datos fiscales (Nombre/Razón Social, NIF/CIF, domicilio fiscal, logo y tipos de impuestos aplicables como IVA e IRPF).

- **Integración con Google Calendar:**

- RF3. Sincronización de calendarios: El sistema debe listar los calendarios del usuario y permitir la vinculación de calendarios específicos con clientes registrados.
- RF4. Lectura de eventos: El sistema debe extraer automáticamente los eventos de los calendarios vinculados, obteniendo fecha, título y descripción para transformarlos en líneas de factura.

- **Gestión de Clientes y Facturación:**

- RF5. Directorio de clientes: El sistema debe permitir el alta, baja y modificación de clientes, asociando cada uno a un nombre de calendario específico.
- RF6. Generación de PDF: El sistema debe generar documentos PDF que cumplan con la normativa legal vigente, incluyendo numeración correlativa, desglose de impuestos y datos de emisor/receptor.
- RF7. Filtro temporal: El usuario debe poder seleccionar un mes y año específicos para agrupar todos los eventos de ese periodo en una única factura por cliente.

- **Histórico y Visualización:**

- RF8. Repositorio de facturas: El sistema debe almacenar un histórico de todas las facturas generadas para su posterior consulta, descarga o eliminación.
- RF9. Panel de previsualización: Antes de generar el PDF, el sistema debe mostrar un resumen de las jornadas detectadas para que el usuario pueda validar o editar la información.

3.2. Requisitos No Funcionales (RNF)

Estos definen las propiedades del sistema y sus limitaciones.

- **Seguridad y Privacidad:**

- RNF1. Seguridad de datos y contraseñas: Las contraseñas de los usuarios no se almacenarán en texto plano bajo ninguna circunstancia, sino que se encriptarán utilizando el algoritmo de hashing BCrypt (generando una huella digital irreversible con "salt"). Además, las credenciales de Google (tokens) se almacenarán de forma aislada en un directorio oculto del sistema operativo para proteger la privacidad del usuario.
- RNF2. Cumplimiento legal: El sistema debe cumplir con el RGPD (Reglamento General de Protección de Datos) al manejar información sensible de clientes y usuarios.

- **Usabilidad y Rendimiento:**

- RNF3. Interfaz intuitiva: Al ser una herramienta para profesionales que buscan ahorrar tiempo, la interfaz debe permitir generar una factura en menos de 3 clics tras el login.
- RNF4. Tiempo de respuesta: La sincronización inicial con Google Calendar no debe demorar más de 5-10 segundos, dependiendo del volumen de eventos.

- **Disponibilidad y Compatibilidad:**

- RNF5. Multiplataforma (si es escritorio): El software debe ser ejecutable en los sistemas operativos más comunes del sector (Windows y macOS).
- RNF6. Persistencia de datos: El sistema debe contar con una base de datos local o en la nube que garantice que los datos de los clientes y el histórico de facturas no se pierdan al cerrar la sesión.

4. Tecnologías previstas.

4.1. Especificaciones del Stack Tecnológico

La arquitectura de AVTech Invoice se ha diseñado buscando un equilibrio entre robustez, escalabilidad y facilidad de mantenimiento. A continuación, se detallan las tecnologías que conforman el ecosistema del proyecto:

- **Back-end y API: El "Motor" en Java:**

El núcleo lógico del sistema se desarrolla íntegramente en Java. Esta elección no es casual: Java proporciona un entorno fuertemente tipado y seguro, ideal para la manipulación de datos financieros y la integración con servicios externos.

- Integración con Google API: Se utilizarán las librerías oficiales de Google para Java (Google Client Library) para gestionar la autenticación OAuth 2.0 y la lectura de eventos a través de Google Calendar API.
- Gestión de Dependencias: Se empleará Apache Maven para centralizar y automatizar la descarga de librerías externas (como conectores de base de datos o generadores de PDF), asegurando que el proyecto sea fácilmente reproducible.
- Generación de Documentos: Se integrará la librería iText o OpenPDF para la lógica de creación de archivos PDF a partir de los datos procesados del calendario.

- **Front-end: Interfaz de Escritorio con JavaFX:**

Para la capa de presentación se ha optado por JavaFX, permitiendo crear una aplicación de escritorio moderna, fluida y multiplataforma.

- Arquitectura FXML: Se separará la lógica de negocio de la interfaz mediante archivos FXML, lo que facilita el mantenimiento del diseño.
- Scene Builder: Se utilizará esta herramienta para el diseño visual de las vistas, asegurando una experiencia de usuario (UX) intuitiva para el técnico freelance.
- Estilización con CSS: Para lograr una estética acorde al sector audiovisual (moderna y limpia), se aplicarán hojas de estilo CSS personalizadas sobre los componentes de JavaFX.

• Gestión de Datos: SQLite (DBMS):

Como sistema de gestión de bases de datos relacionales, se ha seleccionado SQLite por su fiabilidad y amplia documentación.

- Persistencia: SQLite se encargará de almacenar el histórico de facturas, los datos fiscales de los clientes y las preferencias de usuario.
- Integración JDBC: La comunicación entre la lógica en Java y la base de datos se realizará mediante el conector JDBC (Java Database Connectivity), garantizando transacciones seguras y eficientes.
- Integridad de Datos: Se aprovecharán las relaciones de clave ajena (Foreign Keys) para asegurar que cada factura esté correctamente vinculada a un cliente y a un periodo de tiempo real.

• Entorno de Desarrollo (IDE): Eclipse:

Todo el ciclo de desarrollo se centraliza en Eclipse IDE, un estándar en la industria para el desarrollo de aplicaciones empresariales en Java.

- Ecosistema de Plugins: Se hará uso de plugins específicos para la integración con Maven y el soporte de JavaFX, optimizando los tiempos de depuración y compilación del software.

Capa	Tecnología Seleccionada	Función Principal
Lógica / API	Java (JDK 17+)	Procesamiento de datos y comunicación con Google.
Interfaz	JavaFX	Interacción visual con el usuario freelance.
Base de Datos	SQLite	Almacenamiento persistente de facturas y clientes.
Gestión	Maven	Control de librerías y ciclo de vida del proyecto.

Para llevar a cabo el stack que se ha definido (Java, JavaFX, SQLite y Eclipse), es necesario seleccionar librerías específicas que actúen como "puentes" entre el código y los servicios externos (Google y el sistema de archivos PDF).

Para resolver las restricciones de empaquetado de módulos en JavaFX (Fat JAR), la aplicación implementa el patrón Launcher (com.avtech.main.Launcher), aislando el método main principal de la clase que hereda de Application (MainApp.java).

4.2. Integración con el Ecosistema Google.

Como el software depende de Google Calendar y el Login de Google, estas son piezas fundamentales:

- Google Client Library for Java: Es la librería oficial que permite interactuar con los servicios de Google.
- Google OAuth Client Library: Para gestionar el flujo de autenticación "Login with Google" y obtener los tokens de acceso de forma segura.
- Google Calendar API Services: Específicamente para realizar las consultas a los calendarios y filtrar los eventos por fechas y nombres de clientes.

4.3. Capa de Datos (SQLite + Java).

Para conectar Eclipse con la base de datos SQLite, utilizaremos:

- JDBC (SQLite Connector/J): El controlador oficial que permite a Java enviar sentencias SQL a tu base de datos SQLite.

4.4. Generación de Documentos (PDF).

Para transformar los datos de los eventos en facturas profesionales:

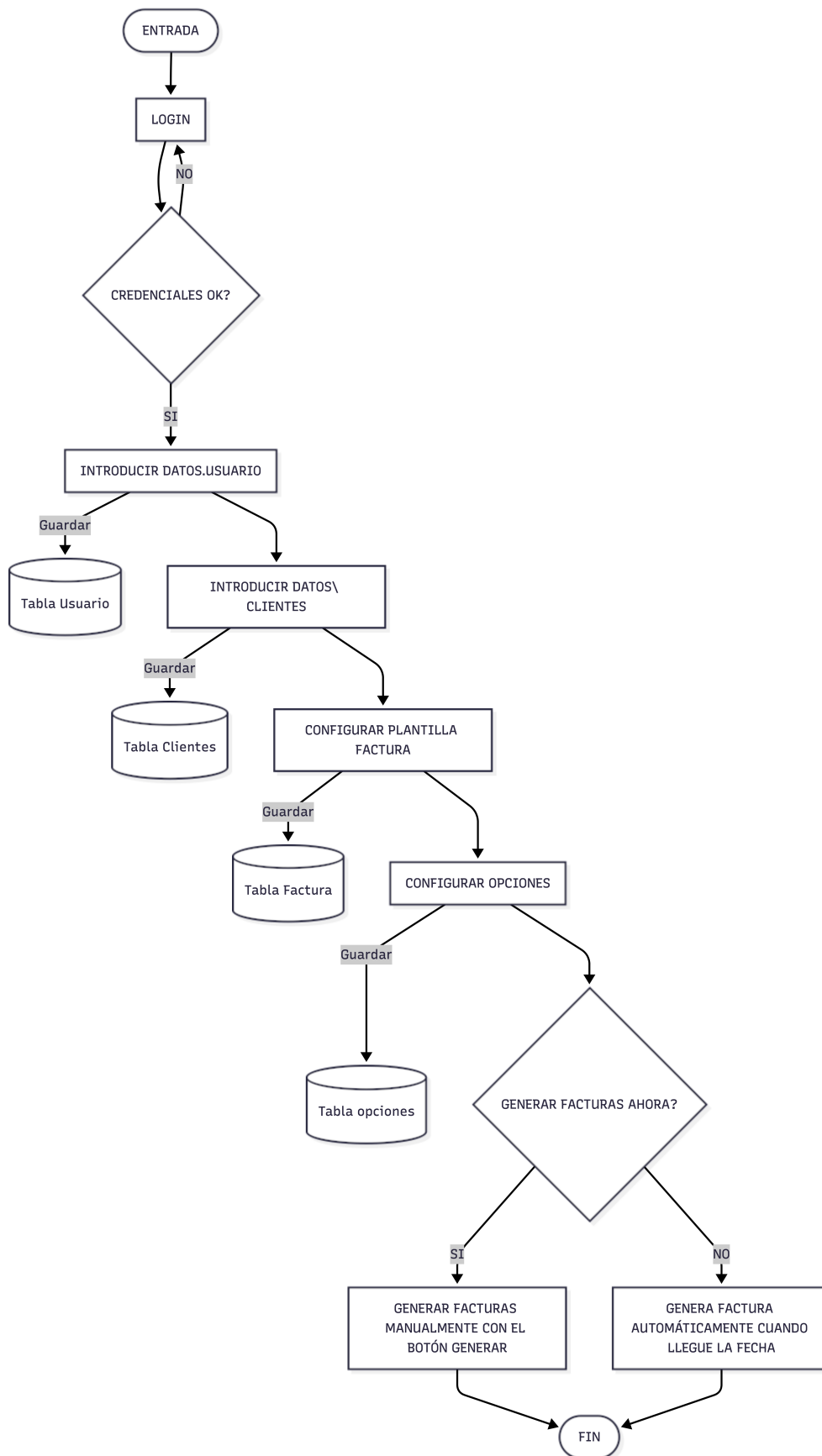
- iText 7 o OpenPDF: Son las librerías estándar en Java para crear PDFs. Permiten definir tablas, insertar tu logo, fuentes personalizadas y generar el documento de forma programática.
- Apache PDFBox: Una alternativa robusta y totalmente gratuita para la manipulación de documentos.

4.5. Interfaz Gráfica (JavaFX).

Para que el software sea visualmente atractivo y funcional en escritorio se usarán las siguientes herramientas:

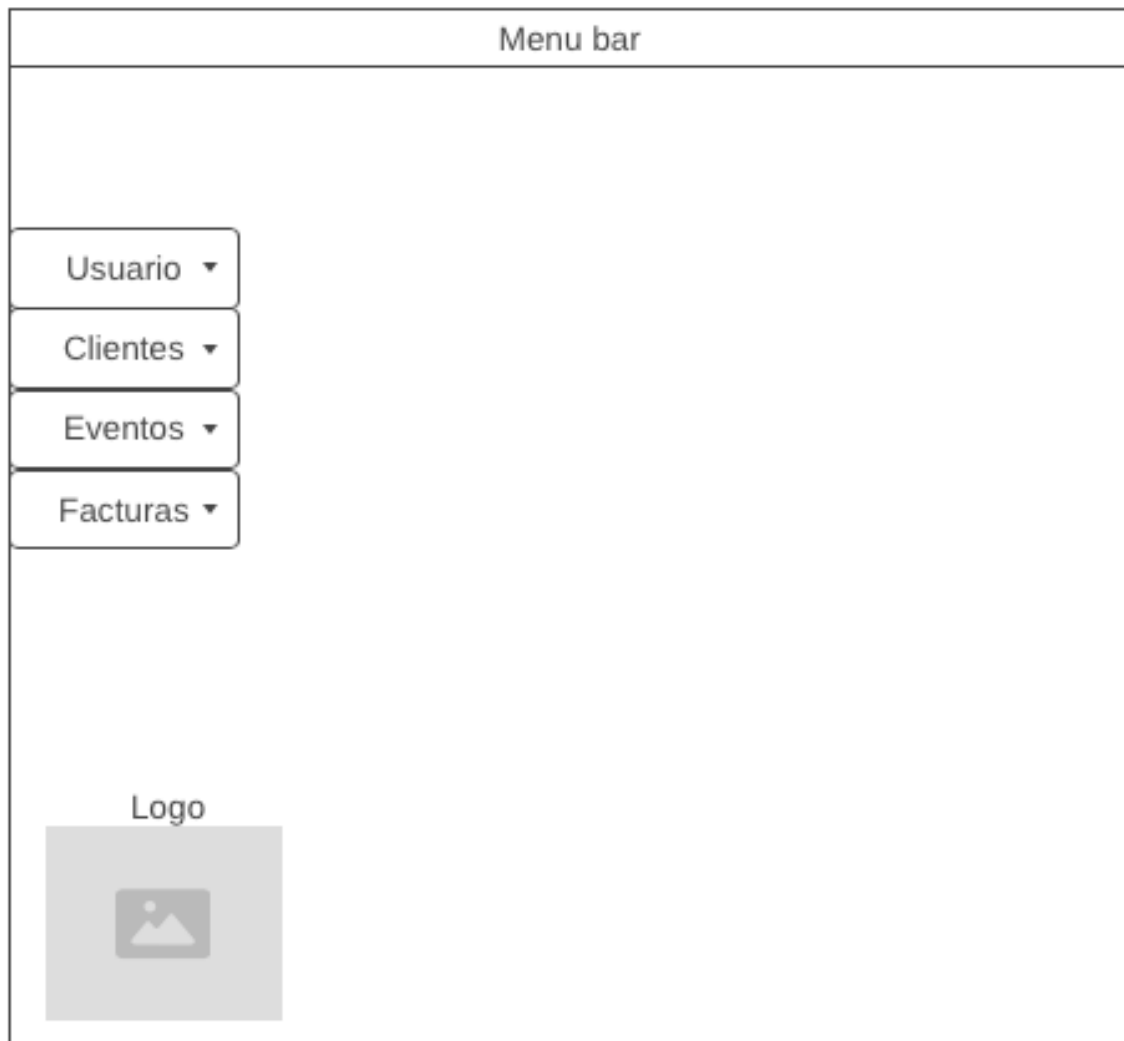
- Scene Builder: Herramienta visual indispensable para diseñar las interfaces .fxml de forma rápida desde fuera de Eclipse.
- ControlsFX: Una librería de componentes adicionales para JavaFX que añade cuadros de diálogo, autocompletado y botones más avanzados.
- CSS para JavaFX: Se utilizarán archivos .css para dar un estilo moderno y profesional a la aplicación (colores, bordes, efectos de hover).

5. Boceto inicial Arquitectura.



6. Esquema visual.

6.1. Esquema de ventana general.



6.2. Esquema de la ventana de Login.

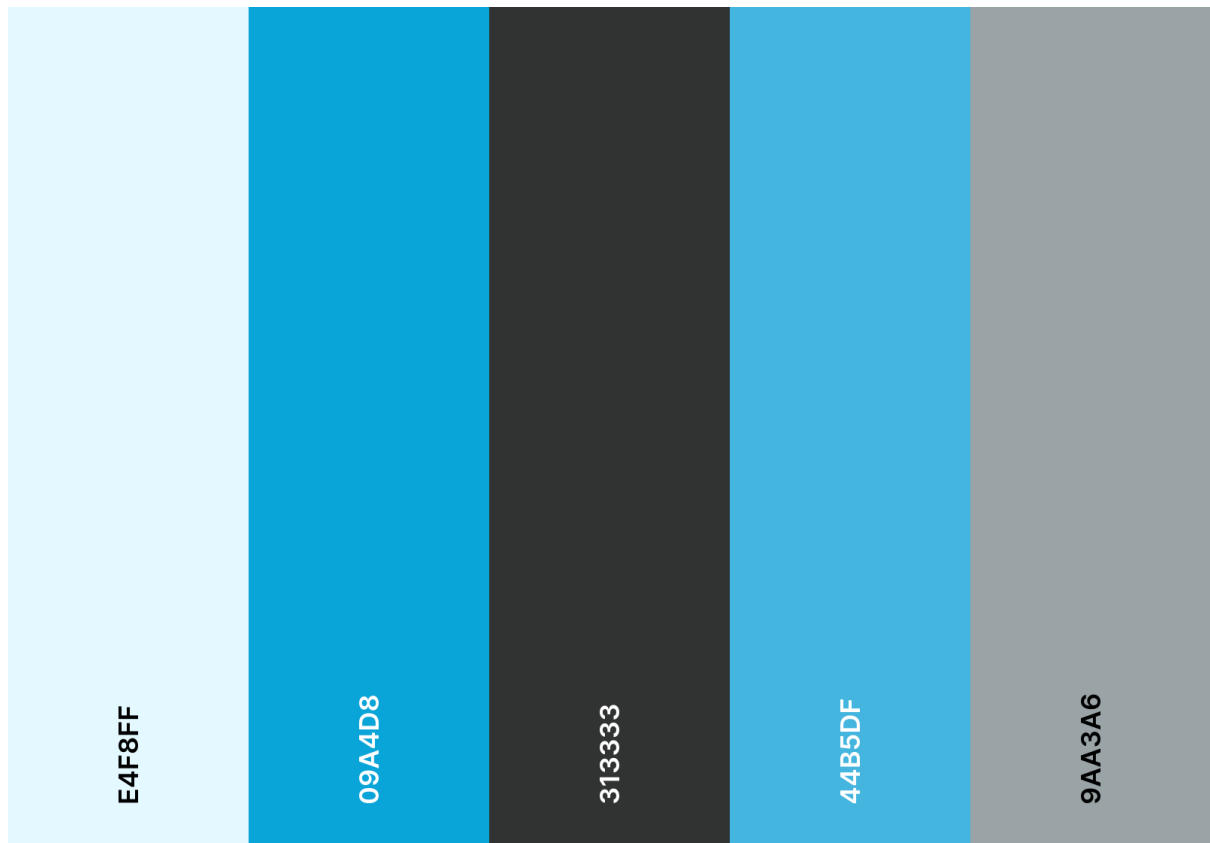
Diagrama de la interfaz de usuario de la ventana de Login:

- Menu bar:** Barra superior que contiene el título "Menu bar".
- Menú lateral:** Se encuentra a la izquierda y contiene cuatro opciones con flechas hacia abajo:
 - Usuario ▾
 - Clientes ▾
 - Eventos ▾
 - Facturas ▾
- Campos de entrada:** Se encuentran en el centro de la interfaz:
 - User Field
 - Password Field
- Logo:** Se encuentra en la parte inferior izquierda, etiquetado como "Logo". Muestra un icono de una imagen dentro de un recuadro gris.

7. Identidad del proyecto.

7.1. Paleta de colores.

Paleta de colores personalizada que se usa para la identidad del proyecto.



Paleta colores

colors

7.2. Logotipo.

El logotipo muestra un calendario y una interfaz de facturación que permite identificar rápidamente el uso para el que está destinado el software, realizado con Gemini Banana y modificado a mano con Adobe Photoshop para retoques, ajuste de detalles e inserción de texto.



7.3.Formato y fuente de texto.

La fuente elegida para el proyecto es System, ya que asegura la compatibilidad con los diferentes sistemas operativos al ser una fuente incluida en el sistema, con sus diferentes variantes según se quiera resaltar el texto o indicar un título o subtítulo.

Los tamaños de fuente que se usan son los siguientes: 18p para títulos, 14p para subtítulos y 11p para el cuerpo del texto.

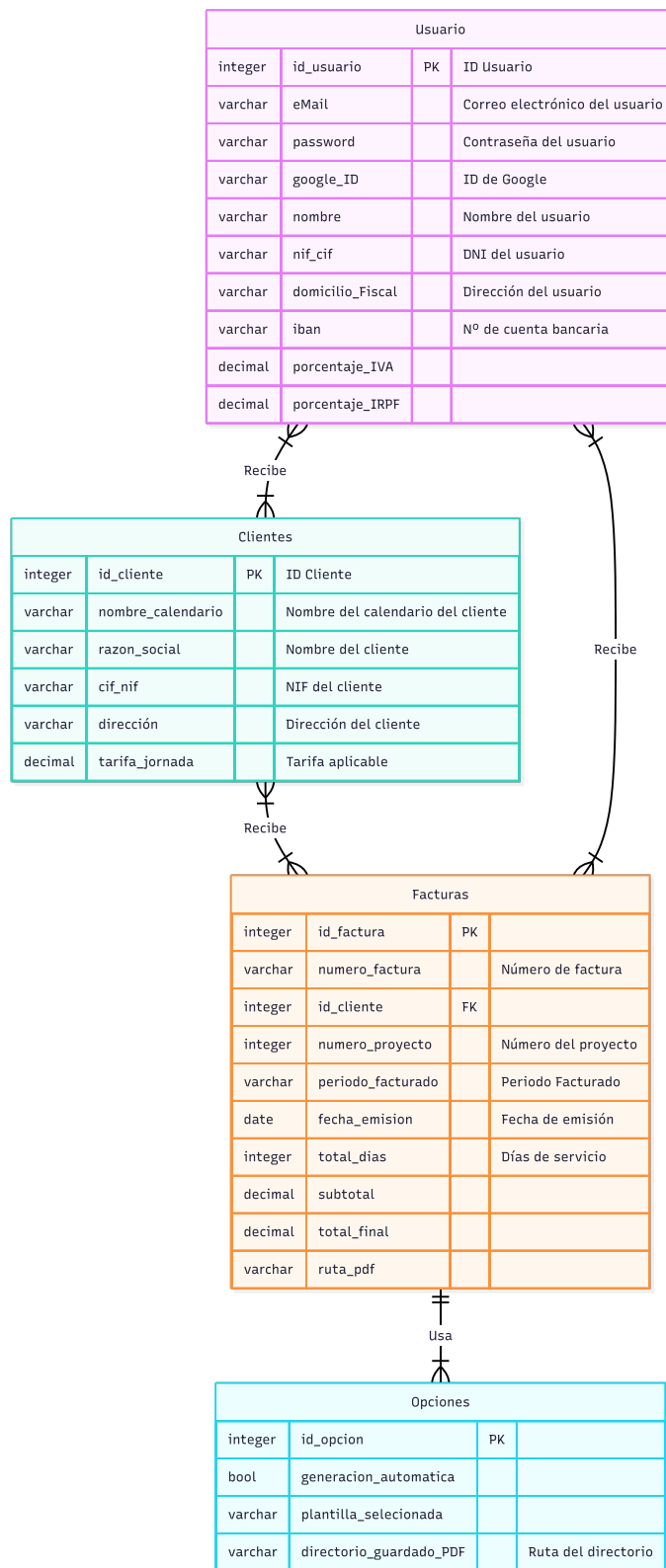
En cuanto al formato de la documentación del proyecto podemos destacar que se realiza con Apple Pages, respetando el espaciado, indentado y estructuras de listas y apartados. Incluyendo un índice interactivo indexado por tabla de contenidos del proyecto.

7.4.Recursos adicionales.

Para generar los diagramas E-R, diagramas de flujo y casos de uso se ha usado mermaid.ai.

Para el el diseño de el wireframe se ha usado wireframe.cc.

8. Diagrama entidad-relación base de datos .



Aclaraciones: *(PK) = Primary Key. *(FK) = Foreign Key.

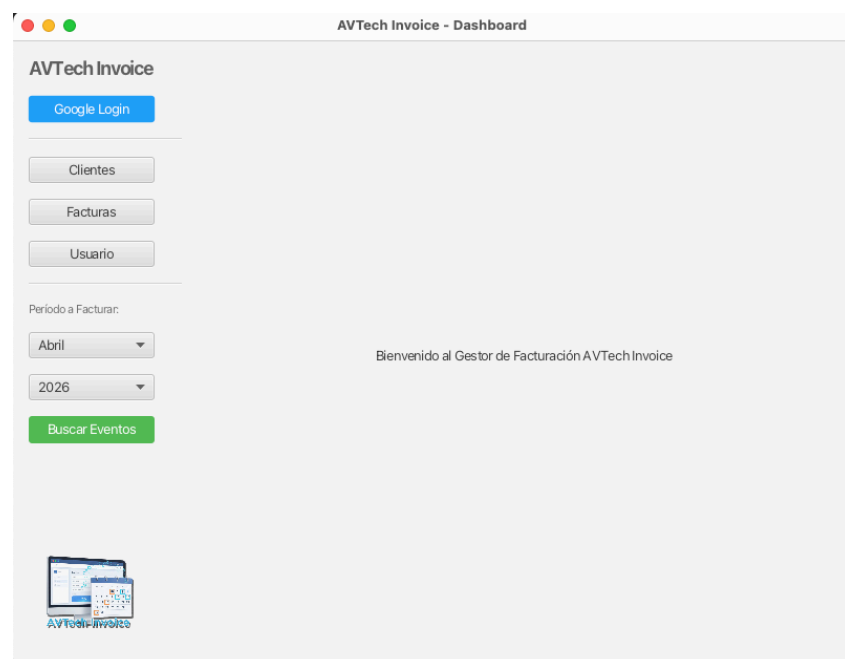
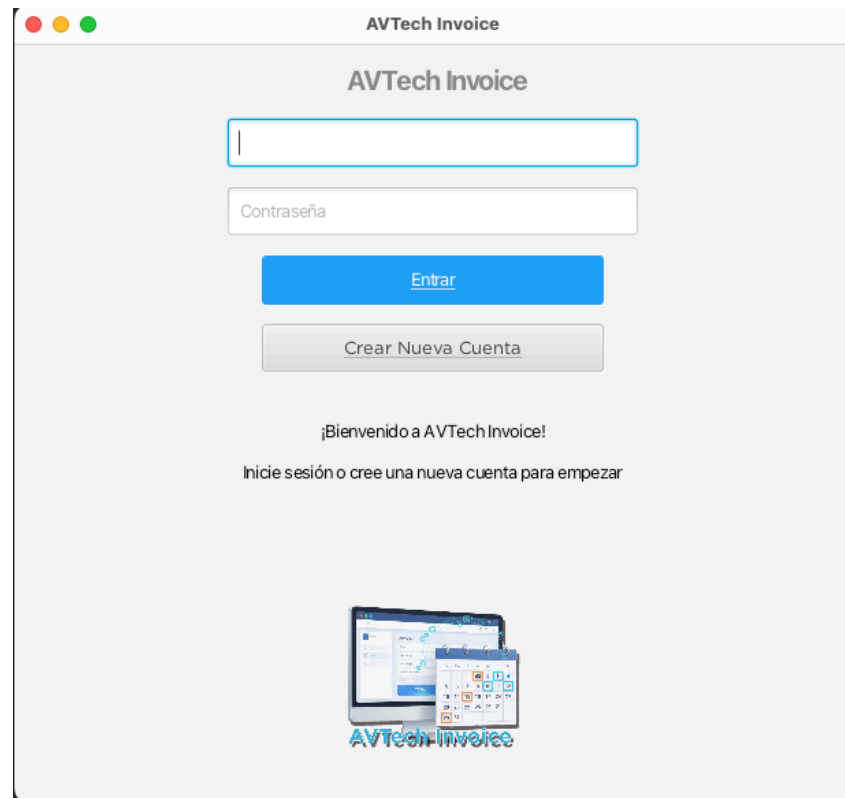
Tablas DB y atributos

Tabla	Atributo 1	Atributo 2	Atributo 3	Atributo 4	Atributo 5	Atributo 6	Atributo 7	Atributo 8	Atributo 9	Atributo 10
Usuario	id_usuario (PK)	eMail	password	google_ID	nombre	nif_cif	domicilio_Fiscal	iban	porcentaje_IVA	porcentaje_IRPF
Cientes	id_cliente (PK)	nombre_calendario	razon_social	cif_nif	dirección	tarifa_jornada				
Facturas	id_factura (PK)	numero_factura	id_cliente (FK)	numero_proyecto	periodo_facturado	fecha_emision	total_dias	subtotal	total_final	ruta_pdf
Opciones	id_opcion (PK)	generacion_automatica	plantilla_seleccionada	directorio_guardado_PDF						

Aclaraciones: *(PK) = Primary Key. *(FK) = Foreign Key.

9. Diseño de la interfaz.

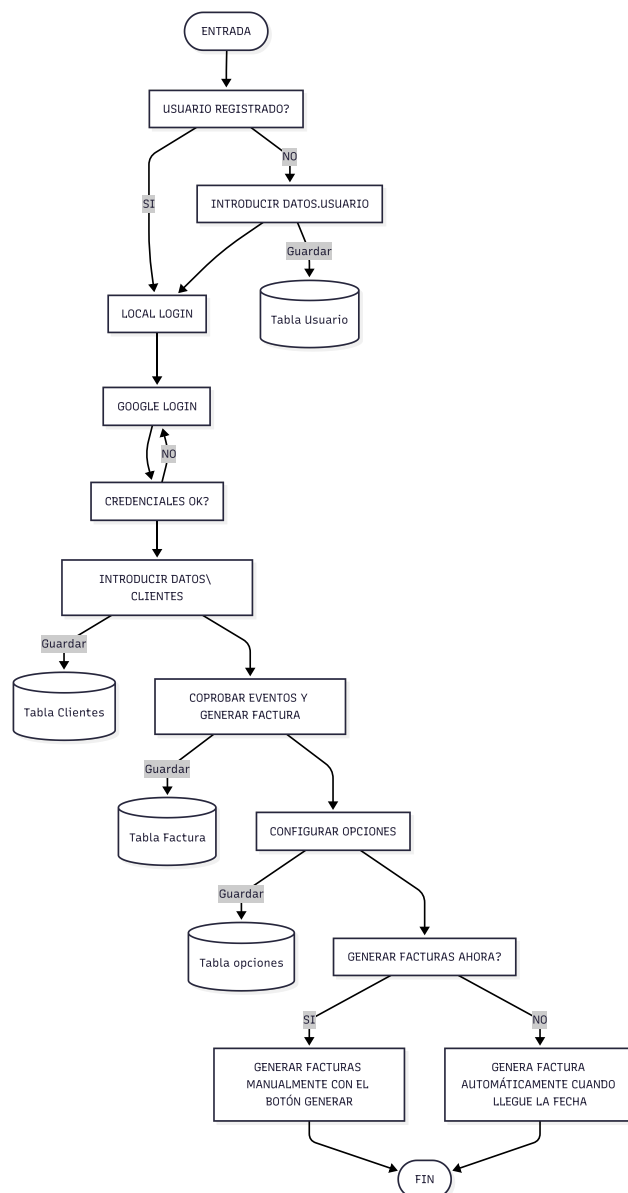
Para que el software sea visualmente atractivo y funcional en escritorio se ha usado Scene Builder, herramienta visual indispensable para diseñar las interfaces .FXML de forma rápida desde fuera del IDE.



10. Diseño de la arquitectura del software.

Para diseñar la arquitectura de AVTech Invoice, se ha seguido un modelo de Arquitectura en Capas (Layered Architecture). Al ser una aplicación de escritorio, este enfoque permite separar la interfaz visual (JavaFX) de la lógica de negocio y el acceso a datos (SQLite y Google API).

La arquitectura final difiere de la original, ya que se han realizados cambios necesarios para la seguridad y el correcto funcionamiento de el software, así como mejorar la UX.



11. Diseño rutas Backend (API REST: endpoints, métodos, recursos).

11.1. Diseño de Servicios y Acceso a Datos (Lógica de Negocio y DAOs)

Al tratarse de una aplicación de escritorio nativa basada en el patrón MVC, el acceso a los datos no se realiza mediante peticiones HTTP a una API REST externa, sino de forma directa e interna mediante el uso de objetos DAO (Data Access Object) para la persistencia en SQLite, y clases de Servicio (**Service**) para la interacción con APIs de terceros (Google) y la generación de archivos físicos.

11.2. Módulo de Autenticación y Gestión de Usuarios (UsuarioDAO y GoogleAuthService).

Gestiona tanto la seguridad local como la vinculación con el ecosistema de Google.

- **Seguridad Local (UsuarioDAO):** Permite el registro e inicio de sesión encriptando las contraseñas mediante el algoritmo **BCrypt**, garantizando que no se guarden en texto plano. Incluye métodos para la creación de perfiles fiscales, actualización de datos (IVA, IRPF, IBAN) y eliminación completa de la cuenta.
- **Autenticación Externa (GoogleAuthService):** Lanza el flujo OAuth 2.0 en el navegador del usuario para obtener los tokens de acceso, los cuales se almacenan de forma segura en un directorio oculto del sistema (~/.avtech/tokens).

11.3. Módulo de Clientes (ClienteDAO).

Controla el almacenamiento persistente de las entidades a las que se les factura.

- **Operaciones CRUD:** Métodos directos a la base de datos para insertar, actualizar, eliminar y listar clientes (insertar, actualizar, eliminarCliente, obtenerTodos).
- **Vinculación:** Incluye métodos específicos de consulta (ej. obtenerTarifaPorNombre, obtenerCifPorNombre) para vincular los nombres de los calendarios detectados en Google con los datos fiscales reales registrados en SQLite.

11.4.Módulo de Calendario (GoogleCalendarService).

Actúa como puente entre la aplicación y la API de Google Calendar.

- **Sincronización (obtenerEventosPorMesAño):** Configura los parámetros de búsqueda para extraer eventos basándose en un mes y año específicos seleccionados por el usuario desde la interfaz gráfica. Los resultados se agrupan automáticamente utilizando el nombre del calendario (cliente).

11.5.Módulo de Facturación (FacturaDAO y FacturaPDFService).

Es el núcleo funcional del proyecto, combinando la persistencia y la creación de documentos.

- **Base de Datos (FacturaDAO):** Almacena el histórico de facturas emitidas, guardando cálculos como bases imponibles, días totales, cliente asociado y la ruta física del documento PDF generado. Permite el borrado del registro (eliminarFactura).
- **Generador de PDF (FacturaPDFService):** Recibe los datos combinados del usuario emisor (sus impuestos configurados) y del cliente. Utiliza la librería iText/OpenPDF para construir un documento legal con cabeceras, tablas de desglose e información de pago, guardándolo físicamente en la carpeta facturas/ del proyecto.

11.6.Gestión Centralizada de la Conexión (DatabaseConnection.java).

- **Finalidad:** Con el objetivo de optimizar el acceso a los datos y cumplir con el principio de reutilización de código (DRY - *Don't Repeat Yourself*), se ha implementado esta clase dentro del paquete com.avtech.model.
- **Funcionamiento:** Actúa como un componente centralizado que gestiona la conexión JDBC con el motor SQLite. En lugar de abrir la base de datos en cada controlador, esta clase proporciona un único método estático getConnection() que es invocado por todos los objetos de acceso a datos (UsuarioDAO, ClienteDAO, FacturaDAO) cuando necesitan realizar operaciones CRUD.
- **Lógica de Persistencia Segura:** Esta clase es la encargada de gestionar la ruta técnica del archivo avtech_data.db. Incluye la lógica necesaria para localizar el directorio oculto en la carpeta personal del usuario (~/.avtech/) y verificar su existencia, garantizando que la aplicación sea portable y que los datos no se pierdan al actualizar o reinstalar el software.

12. Diseño componentes Frontend.

Para el frontend de AVTech Invoice con JavaFX, el diseño debe ser modular. Al tratarse de una aplicación de escritorio, seguiremos el patrón MVC (Modelo-Vista-Controlador), donde cada vista tiene su archivo .fxml (diseño), su controlador .java (lógica de la interfaz) y su hoja de estilos .css.

12.1.Estructura de Contenedores (Layout Principal).

La aplicación utilizará un contenedor raíz (probablemente un BorderPane) para organizar la navegación persistente.

- **MainShell (Contenedor Raíz):**

SideBar (Izquierda): Menú de navegación con botones para Dashboard, Clientes, Calendario e Historial.

ContentArea (Centro): Un StackPane dinámico que intercambia las vistas según la selección del usuario sin recargar toda la ventana.

12.2.Componentes de Vista (Módulos).

Módulo de Autenticación (LoginView.fxml y LoginController.java).

- **Componentes:** Botones con las acciones de “Iniciar sesión” y “Crear nuevo usuario” .
- **Función:** Dispara la autenticación o registro del usuario y, tras el éxito, desbloquea el DashBoard.

Módulo de Gestión de Clientes (ClienteView.fxml y ClienteController.java).

- **ClientTable:** Un TableView que lista: Nombre, NIF, Tarifa/Día y Nombre del Calendario vinculado.
- **ClientForm:** Un formulario para añadir/editar clientes con validaciones de campos (NIF correcto, campos obligatorios).

Panel Principal y Generador (DashboardView.fxml y DashboardController.java).

Este es el corazón de la interfaz.

- **MonthYearPicker:** Selectores para elegir el periodo a facturar.
- **EventList:** Un ListView o TableView que muestra los eventos recuperados de Google Calendar antes de procesarlos.
- **Acción:** Botón “Generar PDF” que envía los datos al backend.

Módulo de Perfil de Usuario (UsuarioView y UsuarioController).

- **Componentes:** Formulario estructurado para la edición de datos fiscales e IBAN bancario.
- **Función:** Permite al usuario mantener actualizada su información fiscal para las facturas, cambiar su contraseña de forma segura y ofrece un control crítico para la eliminación definitiva de su cuenta y borrado en cascada de sus datos, en cumplimiento normativo de privacidad.

Gestión de Archivos Físicos (FacturasView y FacturasController).

- **Visualización:** histórico de facturas y visualización de PDF en lector Pdf por defecto del sistema.
- **Función de Borrado Integral:** Se ha implementado una capa de seguridad en la eliminación de facturas. Al seleccionar un documento y confirmar su eliminación, el sistema no solo borra el registro histórico de SQLite, sino que localiza y destruye automáticamente el archivo PDF físico almacenado en el disco, evitando la acumulación de archivos huérfanos.

Módulo de Registro (RegistroView.fxml y RegistroController.java)

- **Componentes:** Formulario de alta técnica con campos para Email, Contraseña segura, Nombre Fiscal o de Empresa, NIF/CIF, Domicilio Fiscal e IBAN bancario.
- **Función:** Este módulo permite la creación de una cuenta de usuario local de forma segura. Es el componente encargado de procesar la persistencia inicial en la base de datos SQLite mediante el UsuarioDAO , asegurando que la contraseña sea encriptada bajo el algoritmo BCrypt antes de ser almacenada. Una vez registrado con éxito, el usuario es redirigido a la pantalla de acceso para iniciar su flujo de trabajo.

12.3.Componentes de Soporte (Lógica del Cliente).

Para que los componentes visuales funcionen y persistan la información, el frontend se apoya en los siguientes motores internos:

- **Gestión Centralizada de la Conexión (DatabaseConnection.java):** Con el objetivo de optimizar el acceso a los datos y cumplir con el principio de reutilización de código (DRY - Don't Repeat Yourself), se ha implementado esta clase dentro del paquete com.avtech.model.
- **Clases DAO (Data Access Object):** Como ClienteDAO, FacturaDAO y UsuarioDAO, encargadas de gestionar la comunicación directa con la base de datos local SQLite para realizar las operaciones CRUD (Crear, Leer, Actualizar, Borrar) sin necesidad de un servidor externo.

- **Controladores de Estado:** Los datos del usuario activo se mantienen en memoria de forma eficiente mediante variables estáticas centralizadas (ej. `LoginController.usuarioLogueado`), haciéndolos accesibles de forma rápida y segura desde cualquier otra vista sin recargar la base de datos.
- **Clases de Servicio (Service):** Agrupan la lógica de negocio pesada, aislando componentes como `GoogleCalendarService` para interactuar con la API de Google, y `FacturaPDFService` para la maquetación y generación de los documentos físicos en formato PDF.

13. Fase de Desarrollo.

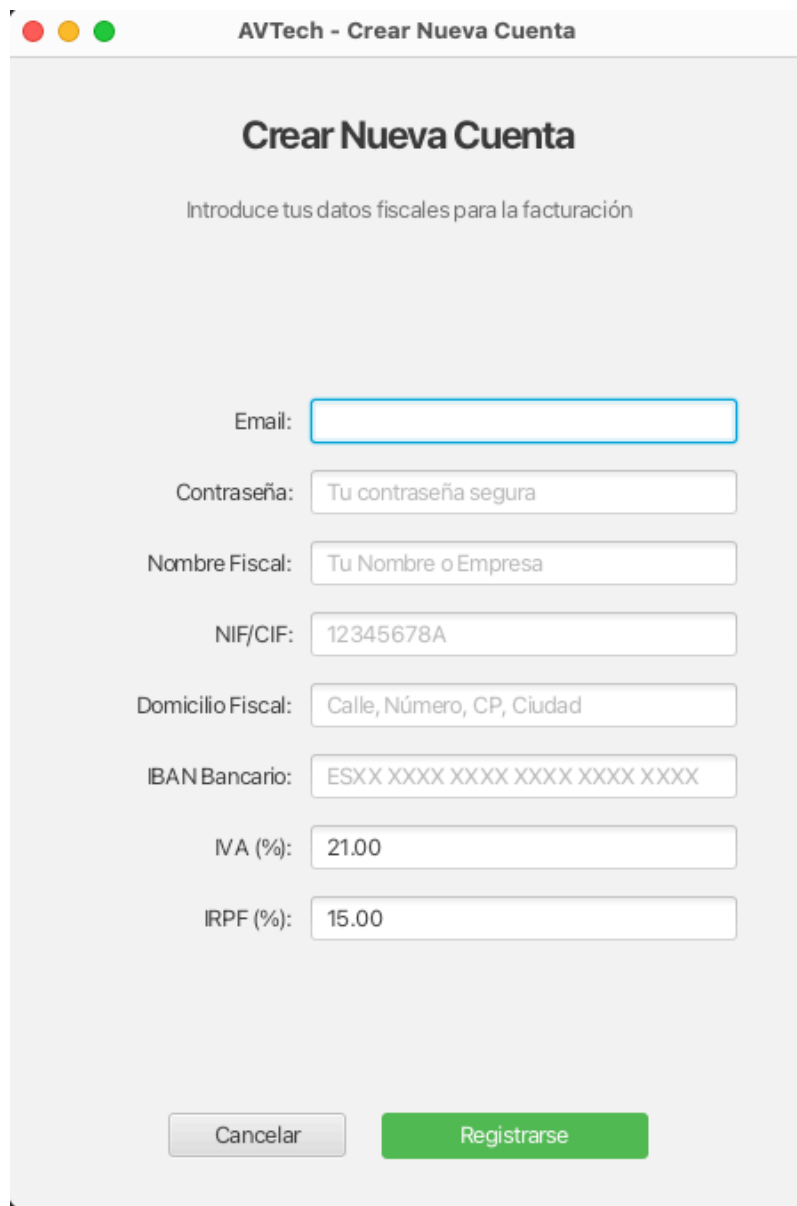
13.1.Enlace a proyecto GitHub.

<https://github.com/SalvaDGS/ Final Proyecto>

13.2.Enlace al video demostrativo de funcionamiento.

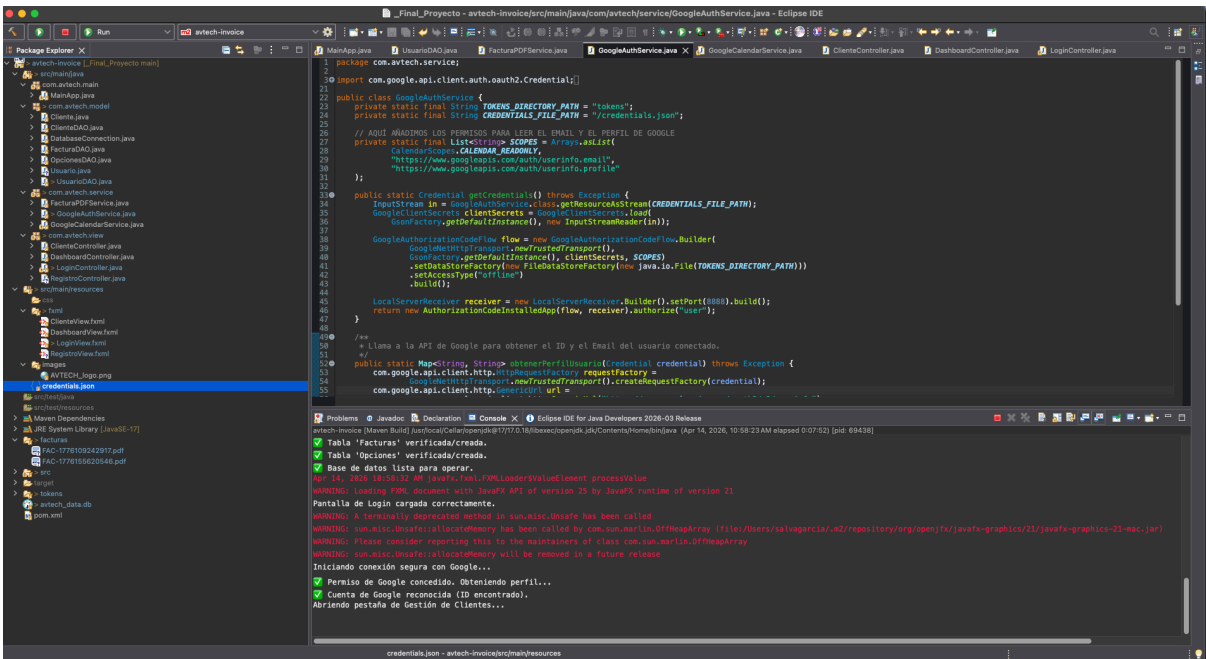
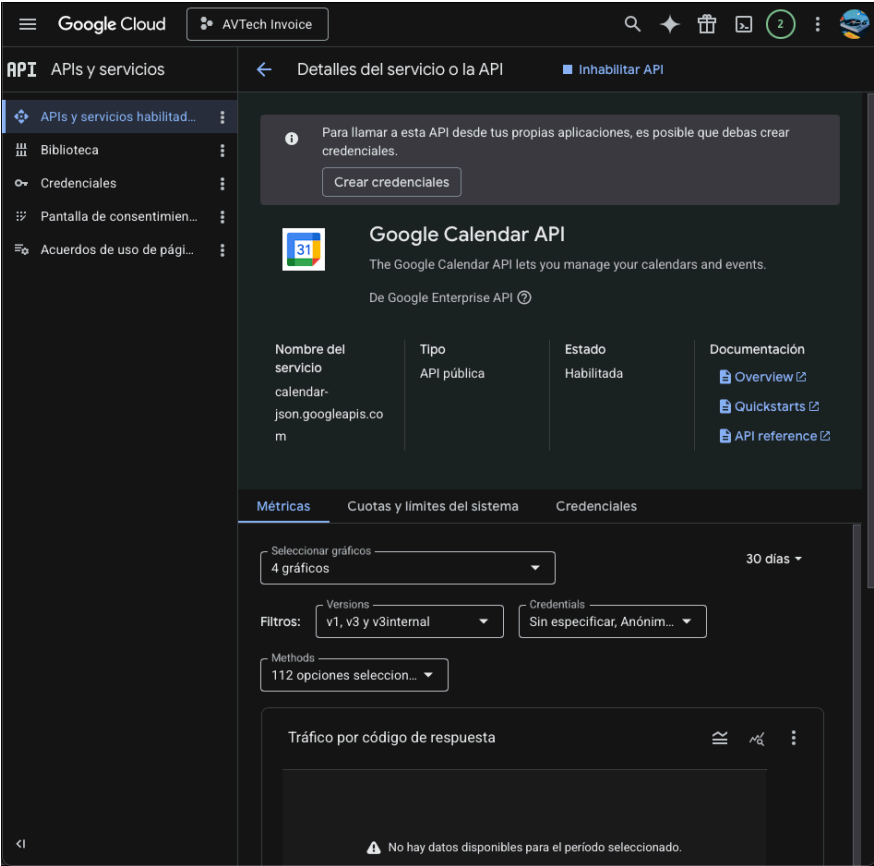
[PI AVTech Invoice Video.mp4](#)

13.3.Creación de un nuevo usuario.



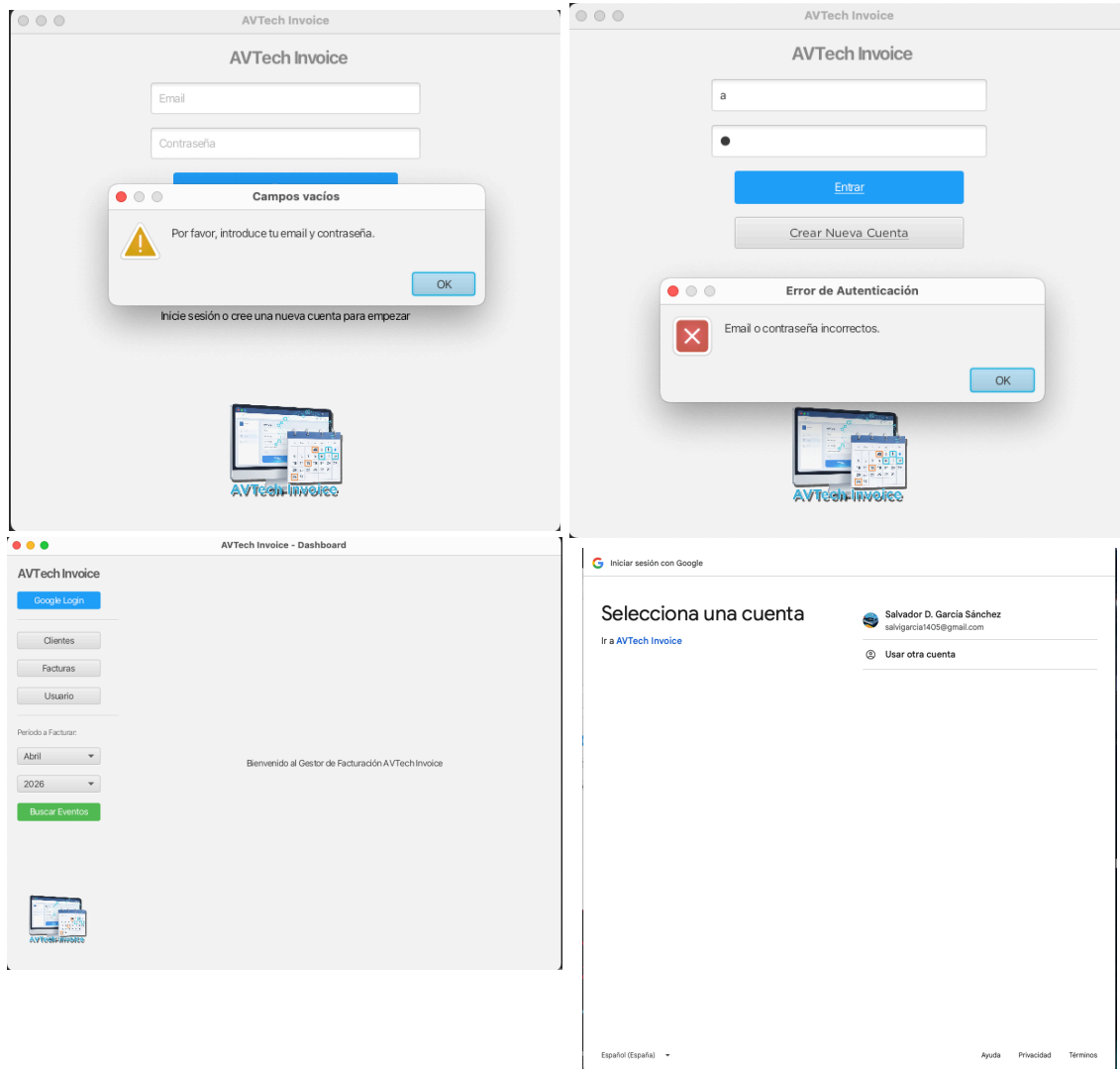
The screenshot shows a web form titled "AVTech - Crear Nueva Cuenta". Below the title is the instruction "Introduce tus datos fiscales para la facturación". The form contains several input fields: "Email:", "Contraseña:" (with placeholder "Tu contraseña segura"), "Nombre Fiscal:" (with placeholder "Tu Nombre o Empresa"), "NIF/CIF:" (with placeholder "12345678A"), "Domicilio Fiscal:" (with placeholder "Calle, Número, CP, Ciudad"), "IBAN Bancario:" (with placeholder "ESXX XXXX XXXX XXXX XXXX XXXX"), "IVA (%):" (with value "21.00"), and "IRPF (%):" (with value "15.00"). At the bottom, there are two buttons: "Cancelar" and "Registrarse".

13.4.Registro de App y suscripción a API Calendar, generando el archivo credentials.json.

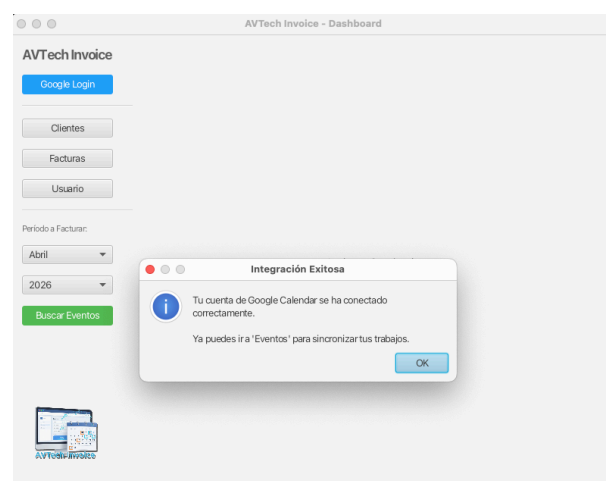


13.5. Proceso de Login

Proceso de login local y con Google. Probando los mensajes de error cuando los campos están vacíos o el usuario es incorrecto.



Received verification code. You may now close this window.



13.6.Ventana cliente.

Creación, modificación y eliminación de clientes.

AVTech Invoice

Google Login

Cientes

Facturas

Usuario

Período a Facturar:

Abril

2026

Buscar Eventos



AVTech Invoice - Dashboard

Gestión de Clientes

Nombre Calendar	Razón Social	NIF/CIF	Tarifa (€)	
Algo Suena S.L.	Algo Suena S.L.	87654321Q	240.0	

Nombre en Calendar:

Ej: Empresa Audiovisual X

Razón Social:

Ej: Empresa Audiovisual X S.L.

NIF/CIF:

12345678A

Dirección:

Calle, Número, CP, Ciudad

Tarifa Jornada (€):

Ej: 250.50

Eliminar Cliente

Limpiar / Nuevo

Guardar Cliente

13.7. Generación de Facturas

AVTech Invoice - Dashboard

Google Login

Cientes

Facturas

Usuario

Período a Facturar:

Abril

2026

Buscar Eventos

CLIENTE: Algo Suená S.L.

Evento 2 AVTI | 2026-04-03T19:00:00.000+02:00 | [Algo Suená S.L.]

Evento de Prueba AVTech Invoice | 2026-04-07T18:30:00.000+02:00 | [Algo Suená S.L.]

CLIENTE: Xmusic S.L.

Evento Plaza Mayor 3 | 2026-04-04T13:30:00.000+02:00 | [Xmusic S.L.]

Generar Factura del Período

AVTech Invoice - Dashboard

Google Login

Cientes

Facturas

Usuario

Período a Facturar:

Abril

2026

Buscar Eventos

CLIENTE: Algo Suená S.L.

Evento 2 AVTI | 2026-04-03T19:00:00.000+02:00 | [Algo Suená S.L.]

Evento de Prueba AVTech Invoice | 2026-04-07T18:30:00.000+02:00 | [Algo Suená S.L.]

CLIENTE: Xmusic S.L.

Evento Plaza Mayor 3 | 2026-04-04T13:30:00.000+02:00 | [Xmusic S.L.]

Generar Factura del Período

Confirmar Generación de Factura

Facturando a: Algo Suená S.L.

Resumen del período:
- Trabajos realizados: 2
- Tarifa por jornada: 240.00€
Base Imponible: 480.00€
IVA (21.0%): +100.80€
IRPF (15.0%): -72.00€
Total a cobrar: 508.80€
¿Deseas registrar esta factura y generar el PDF?

Cancel

OK

Calendar

Hoy

<

>

Abril de 2026

🔍

⚙️

Mes

📅

🔄

+ Crear

Abril de 2026

LUN 30

MAR 31

MIÉ 1 de abr

JUE 2

VIE 3

SÁB 4

DOM 5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

1 de may

2

3

Buscar a gente

Páginas de reserva

Mis calendarios

Gmail

Algo Suená S.L.

Cumpleaños

Tasks

Xmusic S.L.

Otros calendarios

Términos - Privacidad

FACTURA

Salvador García

NIF/CIF: 12345678A

C/ Calle, nº 1, 29190, Málaga

Email: salvargarcia1405@gmail.com

FACTURAR A:

Algo Suená S.L.

NIF/CIF: 87654321Q

C/ Calle, nº 2, 29001, Málaga

Factura N°: FAC-23_04_2026_135955

Fecha de Emisión: 2026-04-23

Concepto	Días	Tarifa/Día	Total
Servicios Audiovisuales del periodo	2	240.00 €	480.00 €

Base Imponible: 480.00 €
IVA (21.00%): 100.80 €
IRPF Retenido (-15.00%): -72.00 €
TOTAL FACTURA: 508.80 €

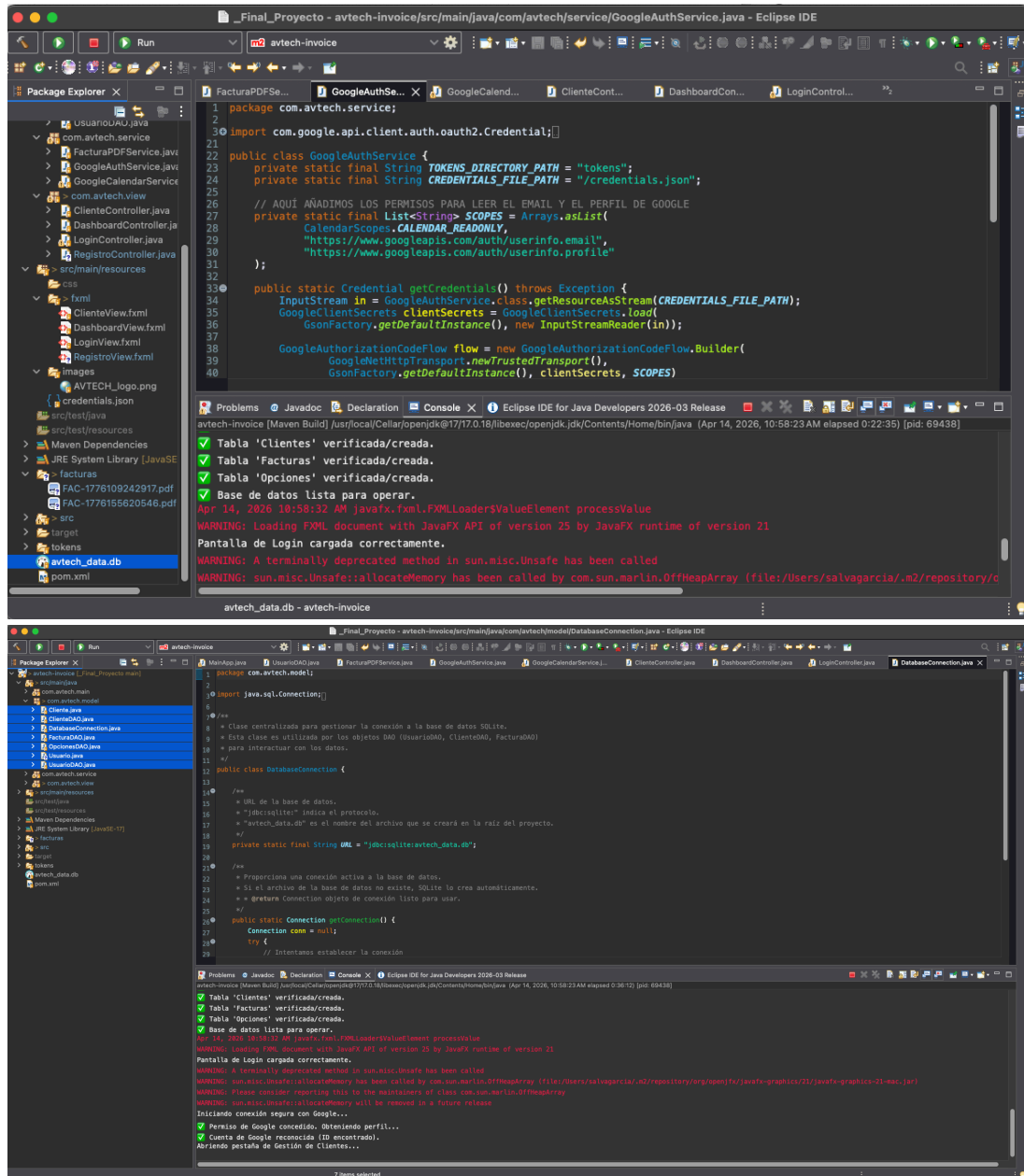
MÉTODO DE PAGO

Transferencia bancaria al siguiente número de cuenta:

IBAN: ES12 1234 1234 1234 1234 1234

13.8.Base de datos.

Comprobación de la base de datos, visualizando los campos y las tablas en Dbeaver:



AVTech Invoice

DBBeaver 25.3.4 - File - /var/folders/s/_1cfz1p014y1g8zpjvpgv9g80000gq/T/.dbeaver-temp10

SQL | Commit | Rollback | Auto | File - /var/foldbeaver-temp10

Database Navigator | Projects | Properties | Diagram

Filter connections by name

- File databases
 - File - /var/folders/s/_1cfz1p014y1g8zpjvpgv9g80000gq/T/.dbeaver-temp10
 - DBBeaver Sample Database (SQLite)

Name

- Bookmarks
- Dashboards
- Diagrams
- Scripts

Properties

Name: File - /var/folders/s/_1cfz1p014y1g8zpjvpgv9g80000gq/T/.dbeaver-temp10

Description: Temporary file datasource for /var/folders/s/_1cfz1p014y1g8zpjvpgv9g80000gq/T/.dbeaver-temp10417664761395452843/lob/avtech_data.db-1776158321392.db

	Table Name	Table Type	Strict typing	Table Description
Tables	Clientes	TABLE	[]	
Views	Facturas	TABLE	[]	
Indexes	Opciones	TABLE	[]	
Sequences	Usuario	TABLE	[]	
Table Triggers				
Data Types				
Driver / ...				

Files - General

Refresh | Save | Revert | 4 items

File - /var/folders/s/_1cfz1p014y1g8zpjvpgv9g80000gq/T/.dbeaver-temp10

DBBeaver 25.3.4 - Usuario

SQL | Commit | Rollback | Auto | File - /var/foldbeaver-temp18

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	id_usuario	email	password	google_id	nombre
1	1	salvigarcia1405@gmail.com	\$2a\$10\$wJp45tMJpiQ2uJdJR58	107769828822411862148	Salvador G

Refresh | Save | Cancel | Export data | 200 | 1

1 row(s) fetched - 0.001s, on 2026-04-29 at 13:32:42

File - /var/folders/s/_1cfz1p014y1g8zpjvpgv9g80000gq/T/.dbeaver-temp18

DBBeaver 25.3.4 - Clientes

SQL | Commit | Rollback | Auto | File - /var/foldbeaver-temp10

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	id_cliente	nombre_calen	razon_social	cif_nif	direccion
1	1	Algo Suen S.L.	Algo Suen S.L.	12345678A	calle n2

Refresh | Save | Cancel | Export data | 200 | 1

1 row(s) fetched - 0.001s, on 2026-04-14 at 11:19:32

File - /var/folders/s/_1cfz1p014y1g8zpjvpgv9g80000gq/T/.dbeaver-temp10

AVTech Invoice

The screenshot displays the DBeaver 25.3.4 - Facturas application window. The interface includes a top toolbar with icons for file operations, SQL execution, and navigation. Below the toolbar, the 'Database Navigator' pane on the left shows a tree view of file databases, with 'File - /var/folders/s/_1cfz1p014y1g8zpjv...' selected. The main workspace is divided into several panes: 'Properties', 'Data', and 'Diagram'. The 'Data' pane is active, showing a table view of data from a SQLite database. The table has five columns: 'id_factura', 'numero_factura', 'id_cliente', 'numero_proyecto', and 'id_cliente'. The data is displayed in a grid format with two rows of data. The first row shows '1' for 'id_factura', 'FAC-1776109242917' for 'numero_factura', '1' for 'id_cliente', and '[NULL]' for 'numero_proyecto'. The second row shows '2' for 'id_factura', 'FAC-1776155620546' for 'numero_factura', '1' for 'id_cliente', and '[NULL]' for 'numero_proyecto'. The bottom status bar indicates '2 row(s) fetched - 0.002s (0.001s fetch), on 2026-04-14 at 11:19:46'.

id_factura	numero_factura	id_cliente	numero_proyecto
1	FAC-1776109242917	1	[NULL]
2	FAC-1776155620546	1	[NULL]

13.9. Empaquetado y Despliegue (Fat JAR y Launcher).

Para convertir el proyecto en un software distribuible e instalable de manera independiente al IDE Eclipse, se ha implementado el patrón Launcher. Al crear una clase principal separada (Launcher.java) que invoca a la clase que hereda de Application (MainApp.java), se superan las restricciones del sistema de módulos de JavaFX. Utilizando el plugin maven-shade-plugin, se compila el proyecto en un "Fat JAR" que engloba el código, las dependencias de Google, SQLite y BCrypt en un único archivo ejecutable, sentando la base para generar instaladores nativos (.app para macOS o .exe para Windows) usando jpackage.

13.10. Creación de Instaladores Nativos (.dmg y .exe) para MacOS y Windows.

Para garantizar que la aplicación sea completamente independiente y pueda ejecutarse en cualquier equipo sin requerir que el cliente final instale el entorno de ejecución de Java (JRE) o las librerías de JavaFX, se ha utilizado la herramienta oficial jpackage (incluida en las versiones modernas del JDK).

El flujo de trabajo requiere generar el "Fat JAR" en el sistema operativo nativo de destino. Este paso es un requisito técnico crítico, ya que Maven debe descargar e incrustar las librerías de renderizado gráfico de JavaFX correspondientes a cada arquitectura de forma estricta (.dylib para macOS y .dll para Windows).

Instalador para macOS (.dmg): Compilando el proyecto en un entorno Mac, se aisló el Fat JAR en una carpeta de empaquetado y se ejecutó el siguiente comando en la Terminal, asignando el icono en formato nativo de Apple (.icns):

```
jpackage \
  --type dmg \
  --app-version 1.0.0 \
  --name "AVTech Invoice" \
  --input empaquetado/ \
  --main-jar avtech-invoice-1.0-SNAPSHOT.jar \
  --main-class com.avtech.main.Launcher \
  --icon ruta/logo.icns
```

Adicionalmente, para evitar bloqueos por parte del sistema de seguridad Gatekeeper de Apple (especialmente restrictivo en arquitecturas Apple Silicon), se aplicó una firma local Ad-Hoc a la aplicación instalada mediante el comando nativo codesign --force --deep --sign - /Applications/AVTech\ Invoice.app, otorgándole los permisos necesarios de ejecución.

Instalador para Windows (.exe): Trasladando el código fuente a un entorno Windows y recompilando el Fat JAR mediante Maven (clean package) en Eclipse para asegurar la inclusión de las librerías .dll nativas de vídeo, se utilizó jpackage apoyado en el motor WiX Toolset. Se aisló el Fat JAR en una carpeta de empaquetado, se añadieron parámetros específicos para integrarlo con el sistema operativo (creación de accesos directos e integración en el menú de inicio) asignando el icono en formato nativo de Windows (.ico), se ejecutó el siguiente comando en PowerShell:

```
jpackage `
  --type exe `
  --app-version 1.0.0 `
  --name "AVTech Invoice" `
  --input empaquetado\ `
  --main-jar avtech-invoice-1.0-SNAPSHOT.jar `
  --main-class com.avtech.main.Launcher `
  --icon ruta/logo.ico `
  --win-shortcut `
  --win-menu
```

El resultado de este proceso son dos instaladores profesionales que contienen su propia Máquina Virtual de Java (JVM) recortada y a medida, ofreciendo una experiencia Plug & Play idéntica a la del software comercial estándar.

13.11.Fase de Pruebas.

Se han realizado todas las pruebas manuales pertinentes, introduciendo valores correctos e incorrectos para ver la respuesta del software así como el correcto funcionamiento de las alertas.

Se han probado de forma manual todas las funciones de la aplicación y se ha observado el resultado y correcto funcionamiento de estas.

Se han probado los instaladores en los sistemas: Windows 11 64 bits, Windows 10 64bits y MacOS 15.7.3.

13.12.Cambios y problemas en la fase de desarrollo

Cambios realizados sobre la idea inicial de proyecto, así como los problemas que se han encontrado en su desarrollo y pruebas.

- **Diseño base de datos:** Basado en el conocimiento adquirido en el período de formación en practicas se ha reestructurado el diagrama E-R, para que el nombramiento de los campos y las relaciones cumplan con las buenas prácticas que se deben llevar a cabo. Además de las modificaciones necesarias para cumplir con las necesidades del proyecto. En este fase del desarrollo se ha decidido cambiar el DBMS de MySQL a SQLite, para mejorar la UX (user experience), al usar SQLite no es necesario que el usuariotenga corriendo un servidor MySQL, ya que SQLite genera un archivo .db en la ubicación del software. Tener que poner a funcionar un servidorMySQL puede suponer un problema para un usuario sin los conocimientos necesarios, al ser las necesidades en cuanto a base de datos bastante básicas para esta aplicación y siendo el estándar para usar en aplicaciones de escritorio se ha optado por SQLite.
- **Uso de Threads:** Inicialmente la aplicación se diseño para que funcionara en un único hilo de ejecución, pero ha sido necesario implementar el uso de hilos secundarios para que la aplicación lleve a cabo procesos como la creación de DB o acceso a credenciales de google de forma paralela a la ejecución de la interfaz para evitar que la aplicación se congele y provoque errores.
- **Cambio de Fuente:** Se ha decidido cambiar la fuente original de Gotham a System, ya que la propuesta inicialmente no es una fuente que se incluya en los SO por defecto, así que puede generar problemas..
- **Implementación de seguridad:** Necesidad de gestionar de forma segura las contraseñas de los usuarios almacenadas en base de datos, así como la ubicación de los archivos que contienen secrets como credentials.json y StoredCredentials.
- **Repositorio GitHub:** Al crear el repositorio inicialmente, no se incluyeron en el .gitignore todos los archivos y carpetas que se deben excluir por seguridad, entre ellos "secrets" pertenecientes a el archivo con los credenciales y los StoredCredentials de usuario. Esto impedía poner el repositorio en público, par esto se excluyeron los archivos de forma manual y se añadieron al gitignore, a pesar de esto, seguía habiendo información sensible expuesta en el repositorio, ya que los commits antiguos en los que se incluía esta información aún eran accesibles en el historial (se ha comprobado el estado de los datos expuestos mediante la plataforma GitGuardian), por lo que finalmente se eliminado el repositorio y creado uno nuevo con las correcciones necesarias.
- **Actualización de la interfaz gráfica:** Cómo mejora a nivel visual y de UX, y siguiendo la recomendación de la tutora del proyecto, se ha rediseñado la interfaz de usuario para que sea más atractiva e intuitiva. También se ha adaptado el formato y el nombre de el recurso logo, para evitar conflictos en los que la imagen no se muestra.

- **Ventana login:** Modificada a una ventana más simple y fácil de interpretar, eliminando el acceso con Google, que pasa a estar dentro del DashBoard y es accesible una vez iniciada sesión local en la aplicación
- **Cambio menús desplegables a botones con Views dentro del DashBoard:** Se ha optado por eliminar los menús desplegables y optar por una interfaz en la que se muestran los botones en la barra lateral y estos abren las distintas Views, haciendo más directo el acceso y reduciendo el número de clicks o acciones necesarias para completar la tarea para la que se ha creado el software.

Cambios que se deberían haber implementado:

Para futuras fases de desarrollo se deberían tener en cuenta los siguientes aspectos que no ha sido posible llevar a cabo por falta de tiempo, se ha priorizado la optimización y revisión de las partes prioritarias funcionales a añadir funciones que no son prioritarias.

- **Factura PDF:** Desglose los servicios prestados, si se requiere, el número de proyecto asociado.
- **Tests:** Implementación de tests automatizados y tests unitarios JUnit. En los que se introducen valores fijos a las entradas y se proporciona la salida esperada para comprobar su correcto funcionamiento, esto ayuda a comprender que se espera del código por parte de otros programadores.
- **Multiusuario:** Actualmente la aplicación está orientada a un único usuario, ya que es una aplicación de escritorio, pero para futuras fases en las que se quiera implementar la posibilidad de que múltiples usuario usen la misma aplicación instalada, habría que vincular el id del Usuario con la información de Clientes, Facturas, etc:
 - Relación de Clientes con Usuario, asignando el id del usuario como clave foránea.
 - Relación de Facturas con Usuario, asignando el id del usuario como clave foránea.

De esta forma se consigue aislar los datos de clientes y facturas para que solo sean visibles por el usuario que los crea. Esto conlleva añadir los campos en las respectivas tablas, así como modificar los métodos e interfaces para que incluyan estos campos en los registros, así como la modificación de las vistas para que solo muestren los campos que estén asociados al id del usuario loggeado.

13.13.Conclusiones.

Una vez finalizado el proyecto se llega a las siguientes conclusiones.

- **Versiones de git:** Se debían haber implementado antes, desde el principio y realizar más commit de versiones o branches para desarrollo, como hemos aprendido en las prácticas, así como una mejor definición inicial de los

archivos a excluir añadidos en el gitignore y comentarios detallados sobre los cambios que se incluyen en cada commit.

- **Documentación de problemas más detallada:** Realizar anotaciones continuas sobre todos los problemas y cambios que surgen, de forma más detallada y especificada en concepto y el momento de realizarlos.
- **Definición inicial del proyecto:** Durante la fase de desarrollo se ha llegado a la conclusión de que había que hacer cambios por necesidades de seguridad, funcionalidad, usabilidad o UX. La idea inicial del proyecto contenía partes imprecisas diseñadas sin los conocimientos necesario, al avanzar en el temario así como en las practicas en empresa se han ido asentando conocimientos y aclarando otros aspectos que se habían interpretado de forma errona. Esto es, según mi opinión, en parte provocado por el nuevo modelo de FP, en el que el proyecto no se inicia una vez terminado el temario y una vez se tienen todos los conocimiento, si no que se inicia en el primer trimestre del curso, cuando aún hay conceptos que no se han visto en profundidad.

13.14.Futuras Líneas de Desarrollo.

El proyecto ha alcanzado un estado de madurez funcional (MVP completo) cumpliendo todos los requisitos iniciales. De cara a futuras versiones y escalabilidad del producto, se plantean las siguientes implementaciones:

- **Automatización de Envíos:** Integración con la API de Gmail (JavaMail) para permitir el envío directo de la factura en PDF al cliente desde la propia pestaña de "Facturas" con un solo clic.
- **Panel de Estadísticas (Dashboard Visual):** Incorporar gráficos de barras y líneas en la pantalla principal usando JavaFX Chart API para visualizar los ingresos mensuales, trimestrales y el volumen de facturación por cliente.
- **Modo Oscuro (Dark Theme):** Añadir un "toggle" en el Perfil de Usuario que cambie dinámicamente las hojas de estilo (CSS) para ofrecer una experiencia visual más descansada, muy demandada en el sector audiovisual.
- **Facturación Automática:** Desarrollar un servicio en segundo plano (Cron Job/ Timer) que utilice la configuración guardada en la tabla Opciones para autogenerar facturas al llegar el último día de cada mes sin intervención manual.
- **Personalización:** Sistema de plantillas predefinidas que aseguran que las facturas cumplan con los requisitos estéticos y legales del profesional.